SWE-EXCELLENCE · DEBUGGING · EMBEDDED C/C++

GDB Investigation Playbook

U	R	D	T	M	B	V
Understand	Root cause	Data and state	Time and tasks	Memory	Boundaries	Verify

26 playbooks · 7 pillars

reverse commands in every pillar that needs them

START HERE

What do I need to learn, and about what?

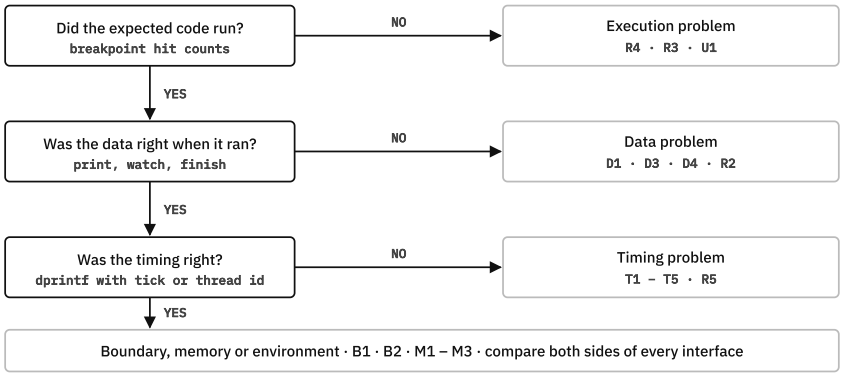
Two axes cover almost every ticket. Choose what you are trying to learn, then what kind of thing you are looking at. The matching playbooks appear below.

Your day-to-day questions

THE QUESTION	PLAYBOOK
What code handles this behavior?	U1
Where should I put this feature?	U4
Where did this value come from? Who changed it?	D1 D4
Why wasn't this code executed?	R4
Why did the wrong code execute?	R3
Why did it happen at the wrong time?	T1 T2 T5
Why did it crash or corrupt state?	R1 M1 M2
Did my change do what I meant?	V1 V2 V3

Before touching the debugger

1. **Write the symptom as one sentence** with the exact text, value, or tick count you saw.
2. **List three possible causes.** "Heartbeat lost" could be never generated, generated wrong, dropped in the queue, received late, rejected by the handler, or timeout set too low.
3. **Pick the observation that separates them.** Which single breakpoint or watchpoint would be true under one cause and false under the others?
4. **Make one observation. Cross off causes.** Repeat. If nothing got crossed off, the observation was a bad one.



Each question needs one observation to answer. Answer them in this order because every later question assumes the earlier ones came back clean.

THE LIBRARY

Playbooks

Each card has the prompts to answer first, a paste-ready GDB block, how to read the result, how to go backwards when you overshoot, and what to write down. Angle brackets like `<Function>` are yours to fill in.

U1 Execution path discovery

"What code handles this?" "What actually runs when I do X?"

ASK YOURSELF FIRST

- What input can I trigger myself: a message, a command, a button, a timer?
- Which function do I know for sure is involved, even loosely?
- Is the path static (direct calls) or dynamic (callbacks, virtuals, function pointers, a registry)? Dynamic is where reading fails and running wins.

READING THE RESULT

The first frame that is framework code (scheduler, dispatcher, `std::function`) marks where your application path begins. The frame just inside it is your entry point.

IF YOU OVERSHOOT

Overshot the interesting call? With recording on: `reverse-finish` to the call site. Without: `tbreak` it and rerun.

WRITE THIS DOWN

Paste the `bt` into your journal as a call path with file:line for each frame. It is your map of this feature.

GDB BOILERPLATE

```
$ gdb --args ./app <args>
(gdb) start                        # stops at main
(gdb) break <KnownFunction>
(gdb) continue                     # now trigger X
(gdb) bt                           # who called it, innermost first
(gdb) bt -full 3                   # top 3 frames with their locals
(gdb) up                           # climb to the caller, then:
(gdb) info locals
(gdb) info frame

# Log the path downstream without stopping (narrow the regex!):
(gdb) rbreak ^Heartbeat::
(gdb) info breakpoints             # note the number range, e.g. 4-19
(gdb) commands 4-19
> silent
> bt 1
> continue
> end
(gdb) continue
```

U2 Runtime architecture discovery

"What are the pieces of this system?" "Which tasks and queues exist?"

ASK YOURSELF FIRST

- Who creates tasks or threads, and with what entry function and priority?
- Which queues, semaphores or mailboxes connect them?
- Which task blocks on what, right now?

READING THE RESULT

Build one table: task name, entry function, priority, what it blocks on, what it posts to. After one run of startup you will know more than the source tree told you.

IF YOU OVERSHOOT

Missed startup? Restart and set the breakpoints before `run`. Create-calls are one-shot events.

WRITE THIS DOWN

The task table. It becomes the legend for every later timing or deadlock investigation.

GDB BOILERPLATE

```
# Linux threads
(gdb) break pthread_create
(gdb) commands
> silent
> info args
> bt 3
> continue
> end
(gdb) run
(gdb) info threads
(gdb) thread apply all bt 8

# RTOS: swap in your kernel's create call
(gdb) break xTaskCreate           # or osThreadNew, taskSpawn, ...
(gdb) commands
> silent
> printf "task %s prio %d\n", pcName, uxPriority
> continue
> end
```

U3 Which instance am I looking at?

"There are 100 of these." "Which Controller handles my case?"

ASK YOURSELF FIRST

- How many instances exist, and who creates them?
- What identifies mine: an id, a name, a config value, an owner pointer?
- Who owns it, and what is its configuration?

READING THE RESULT

The address is the identity. The class name is ambiguous, the address is not. From here on, condition everything on `this == address` and the other 99 instances disappear.

IF YOU OVERSHOOT

Addresses change on every run. Heap addresses are stable run to run only when ASLR is off (GDB turns it off by default).

WRITE THIS DOWN

A small table: address, id or name, owner, config. Note it is for this run only.

GDB BOILERPLATE

```
# 1. Enumerate instances as they are born
(gdb) break Controller::Controller
(gdb) commands
> silent
> printf "ctor this=%p\n", this
> bt 4
> continue
> end
(gdb) run

# 2. Pin to ONE instance by address from then on
(gdb) break Controller::handle if this == 0x5555556a4eb0
(gdb) continue
(gdb) p *this
(gdb) p *this->owner_
(gdb) ptype this
(gdb) ptype/o Controller          # field offsets and padding
(gdb) info vtbl this
```

U4 Find the feature insertion point

"Add behavior when heartbeat status changes." "Where do I put this?"

ASK YOURSELF FIRST

- Where does the triggering input enter the system?
- Where is the decision made, and where is its result consumed?
- Where is the state stored?
- What is the smallest frame where I have every input I need and can still affect the output?

READING THE RESULT

If your intended expression prints the right value at this frame, this is a viable hook. If you have to reach through three pointers to build it, look one frame up or down.

IF YOU OVERSHOOT

Moved past the zone? `up` and `down` are free. With recording, `reverse-step`.

WRITE THIS DOWN

A short design note: "Input at A:line, decision at B:line, output at C:line, hook at B."

GDB BOILERPLATE

```
(gdb) break <where the input enters>     # receiver, ISR handler, API
entry
(gdb) run
(gdb) bt                                    # call path A
(gdb) break <where the output leaves>       # transmit, actuator
write
(gdb) continue
(gdb) bt                                    # call path B
# The frames common to both backtraces are your candidate zone.
(gdb) up
(gdb) info locals                           # do I already have
everything here?
(gdb) p <the condition you plan to write>   # evaluate your new code
BEFORE writing it
(gdb) call <existing_helper>(<args>)        # does the helper return
what I assume?
```

R1 Crash: segfault, abort, assertion

"SIGSEGV in ..." "Assertion failed" "terminate called"

ASK YOURSELF FIRST

- Which signal? Which address was touched: near zero, garbage, or a stack address?
- Which frame owns the bad pointer? It is often not frame 0.
- Who set that pointer last?

READING THE RESULT

A fault address like `0x18` means a null object with a member at offset 24. Confirm with `ptype/o`. A fault in libc usually means your code passed it something bad one or two frames up.

IF YOU OVERSHOOT

Crash site is not bug site. Rerun with `watch -l ptr` to catch the write that broke it, or use rr and `reverse-continue` from the crash.

WRITE THIS DOWN

Save `bt full` to a file. Note: signal, fault address, the owning frame, and who last wrote the pointer.

GDB BOILERPLATE

```
(gdb) run                                  # or: gdb ./app core
(gdb) bt full
(gdb) p $_siginfo._sifields._sigfault.si_addr  # faulting address
(Linux)
(gdb) frame 1
(gdb) info locals
(gdb) p *ptr
(gdb) x/6i $pc-12                            # instructions around
the fault
(gdb) info registers

# abort() or assert(): climb out of libc to your code
(gdb) bt
(gdb) frame function <YourFunction>

# C++ exceptions and aborts, stop at the source
(gdb) catch throw
(gdb) catch signal SIGABRT
```

R2 Wrong result: first point of divergence

"Expected 42, got 37." "Output is off."

ASK YOURSELF FIRST

- What are the stages between input and output?
- What is the last stage where the value is definitely right, and the first where it is definitely wrong?
- Is the wrongness a bad computation, a stale input, or an overwrite?

READING THE RESULT

You are done when you have a stage where input is right and output is wrong. Everything upstream is innocent.

IF YOU OVERSHOOT

`reverse-finish`, `reverse-next`. Without recording: `ignore` the first N-1 hits and rerun.

WRITE THIS DOWN

A stage table: stage, value, right or wrong. The culprit is the row where it flips.

GDB BOILERPLATE

```
# Bisect the pipeline. Condition on the bad case so you stop only
there.
(gdb) break stageB if input_id == 1337
(gdb) run
(gdb) p value                        # right or wrong at B's entry?
(gdb) finish                         # prints "Value returned is $1 =
..."
(gdb) p $                            # that value again
# Halve the interval: right at B, wrong at D? Look at C next.
(gdb) tbreak stageC
(gdb) continue
(gdb) display value                  # shown after every stop from now
on

# Or: stop on the first wrong write wherever it happens
(gdb) watch -l obj->result if obj->result != 42
```

R3 Wrong code ran

"Wrong handler." "State machine went to the wrong state." "Wrong branch."

ASK YOURSELF FIRST

- What does the condition evaluate to, and is it correct?
- If the value is right but the dispatch is wrong: virtual call, function pointer, or table lookup?
- Who made the choice: this function or its caller?

READING THE RESULT

If forcing the value fixes the symptom, the bug is whoever set `state_`. Go to D1 and watch it. If the value was right, the bug is in the lookup.

IF YOU OVERSHOOT

`set var` is an experiment, not a fix. Print first (`p state_`), so `$N` remembers the old value. Or `checkpoint` before experimenting.

WRITE THIS DOWN

"At file:line, `state_` was X, expected Y. Forcing Y made the test pass."

GDB BOILERPLATE

```
(gdb) break <function with the branch>
(gdb) run
(gdb) info locals
(gdb) p <the condition expression>
(gdb) p/d state_                     # enum as a number, in case of
mismatch
(gdb) p handler_                     # function pointer: where does
it point?
(gdb) info symbol handler_
(gdb) set print object on
(gdb) p *obj                         # real dynamic type
(gdb) info vtbl obj

# Experiment: does forcing the expected value fix the downstream
behavior?
(gdb) set var state_ = <Expected>
(gdb) next

# Who dispatched into the wrong handler?
(gdb) tbreak <WrongHandler>
(gdb) continue
(gdb) bt
```

R4 Expected event never happened

"Heartbeat never sent." "Callback not called." "Task never wakes."

ASK YOURSELF FIRST

- What is the chain of calls that should have happened, in order?
- Where did it last definitely work?
- Is the missing step blocked, never reached, filtered by a guard, or run but silent?

READING THE RESULT

Hit counts form a ladder: 10, 10, 10, 0. The break is between the third and fourth rung. Absence of execution has a place; find the place.

IF YOU OVERSHOOT

The hit count tells you the iteration. Rerun with `ignore` to land on the last good one.

WRITE THIS DOWN

The ladder: link, hit count. Paste it into the ticket.

GDB BOILERPLATE

```
# Put a breakpoint on every link in the expected chain, in order
(gdb) break Sender::build
(gdb) break Sender::enqueue
(gdb) break Dispatcher::deliver
(gdb) break Receiver::onHeartbeat
(gdb) run                           # then trigger the scenario and
let it run
^C                                  # interrupt after a while if it
hangs
(gdb) info breakpoints              # read the "already hit N times"
column

# The first link with fewer hits than the one before it is the gap.
# Now look inside the link BEFORE the gap:
(gdb) break Sender::enqueue
(gdb) run
(gdb) next                          # watch for an early return
(gdb) p <each guard condition>
(gdb) thread apply all bt           # or is something simply blocked?
```

R5 Intermittent failure

"Works 99 times." "Only under load." "Only after a few minutes."

ASK YOURSELF FIRST

- Can I make it happen more often: a loop, stress, one core, a fixed seed?
- What differs between runs: timing, input order, uninitialized memory, ASLR?
- Does a breakpoint hide it? Then use `dprintf`, not `break`.

READING THE RESULT

Compare a failing log with a passing log. The first line that differs is the first clue. If adding output "fixes" it, you are likely looking at a race or undefined behavior.

IF YOU OVERSHOOT

rr is the real answer here: record a failing run once, then reverse as much as you like.

WRITE THIS DOWN

Failure rate before and after any change. "1 in 40" becoming "0 in 500" is a result.

GDB BOILERPLATE

```
# Unattended loop until it fails; keep the evidence
$ for i in $(seq 1 500); do
>   gdb -batch -ex run -ex 'bt full' --args ./app <args> > run.$i.log
2>&1
>   grep -q 'SIGSEGV\|Assertion' run.$i.log && { echo "failed on run
$i"; break; }
> done

# Bugs hidden by GDB turning ASLR off:
(gdb) set disable-randomization off

# Observe without stopping
(gdb) dprintf Worker::step,"tid=%d seq=%d\n", $_thread, seq
(gdb) catch signal SIGSEGV
$ ulimit -c unlimited                 # leave a core for the failure
you can't attach to

# Race suspected? Replay it deterministically
$ rr record --chaos ./app
```

D1 Who changed this value?

"Why is mode SAFE?" "Where did 523 come from?" A variable named in 100 places.

ASK YOURSELF FIRST

- What value is bad, and what value would be good?
- Which instance, by address?
- Is the write I will catch the first bad one, or did an earlier write set it up?

READING THE RESULT

The backtrace at the stop is the answer to "who touched it". Read the arguments of the writer frame: was the value wrong when it arrived? If so, the writer is innocent. Move up the chain, or to D4 if a message queue sits in between.

IF YOU OVERSHOOT

With recording or rr, put the watch on the final bad value and `reverse-continue` to land on the last write. No guessing which hit.

WRITE THIS DOWN

Writer frame, its arguments, the old and new values, and the path above it.

GDB BOILERPLATE

```
(gdb) break <line after the object exists>
(gdb) run
(gdb) p &obj->mode_                        # the address IS the
identity
(gdb) watch -l obj->mode_                  # every write
(gdb) watch -l obj->mode_ if obj->mode_ == 3     # only the
interesting one
(gdb) rwatch -l obj->mode_                 # reads
(gdb) awatch -l obj->mode_                 # reads and writes
(gdb) continue
# Watchpoint 2: Old value = 1 / New value = 3, stopped in the writer
(gdb) bt
(gdb) frame 1
(gdb) info locals

# If the condition cannot see your locals, use the raw address:
(gdb) watch *(int*)0x7ffe4d2c if *(int*)0x7ffe4d2c == 3
(gdb) info watchpoints
```

D2 Where does this value go?

"What consumes heartbeat status?" "What breaks if I change this?"

ASK YOURSELF FIRST

- Who reads this field, and under what conditions?
- Does it get copied into something else (a message, a cache, a log)?

READING THE RESULT

The distinct frame-1 callers in the log are the consumers. If you only saw one reader, note whether your scenario reached the others at all.

IF YOU OVERSHOOT

Hardware watchpoints are limited (often 4). Delete finished ones: `delete 3`.

WRITE THIS DOWN

Consumer list with file:line. This is your impact map for a change.

GDB BOILERPLATE

```
(gdb) break <line after the object exists>
(gdb) run
(gdb) rwatch -l obj->status_
(gdb) commands
> silent
> bt 3
> continue
> end
(gdb) continue                     # a list of every reader, in order

# Then find the copy: where does the value end up?
(gdb) awatch -l obj->status_       # spot copies by value, print and
compare
(gdb) p/x obj->status_
```

D3 How did A become B? (transformation chain)

"Sensor reads 4095 but display shows -1." "Scaled value is off."

ASK YOURSELF FIRST

- What are the stages (scale, filter, convert, clamp) and what unit does each use?
- Where could a cast, truncation, sign change, or integer promotion happen?
- Is each stage correct in isolation, and wrong in combination?

READING THE RESULT

The stage where the value stops matching your hand calculation is the stage to read. Common culprits: signed versus unsigned, integer divide before float multiply, 16-bit truncation, a clamp that is applied twice.

IF YOU OVERSHOOT

`reverse-step` into the stage. Or rerun with a conditional break on that stage only.

WRITE THIS DOWN

A table: stage, input, output, units, hand-computed expected.

GDB BOILERPLATE

```
# One labelled trace line per stage, same format, then compare
(gdb) dprintf Sensor::read,"S1 raw=%d\n", raw
(gdb) dprintf scale,"S2 scaled=%f\n", out
(gdb) dprintf toFixed,"S3 fixed=%d\n", out
(gdb) dprintf clampTo,"S4 clamped=%d\n", out

# Inspect types where a surprise is likely
(gdb) ptype expr
(gdb) whatis (a * b)
(gdb) p sizeof(x)
(gdb) p/x x
(gdb) p/t x
(gdb) p (short)x
(gdb) finish                        # shows "Value returned" for any
function
```

D4 Message flow trace

"Message lost." "Command not received." "Wrong destination."

ASK YOURSELF FIRST

- Which hop failed? Use the list below.
- Break at the sender, not the receiver. After a queue the receiver's stack shows only the dispatcher; the sender is gone.
- Is there a message id or sequence number I can follow?

READING THE RESULT

Compare the POST and HANDLE streams. A POST without a HANDLE: dropped or queued but never pumped. A HANDLE with a different value: changed in transit. Also avoid calling container methods like `size()` inside `dprintf`; use fields or pretty printers.

IF YOU OVERSHOOT

`reverse-continue` from a bad HANDLE lands on the matching POST if you are recording.

WRITE THIS DOWN

The failing hop. The ten categories: not created · created wrong · not sent · wrong destination · queued wrong · dropped · delayed · received wrong · dispatched wrong · handled wrong.

GDB BOILERPLATE

```
# Stop at the PRODUCER, filtered on the bad payload
(gdb) break Bus::post if m.value < 0
(gdb) run
(gdb) bt                            # frame 1 is the sender
(gdb) frame 1
(gdb) p *this                       # why did it build this message?

# Follow one message through every hop without stopping
(gdb) dprintf Bus::post,"POST   type=%d val=%d\n", m.type, m.value
(gdb) dprintf Bus::pump,"PUMP\n"
(gdb) dprintf Tracker::onMessage,"HANDLE type=%d val=%d\n", m.type,
m.value
(gdb) set logging file msgs.log
(gdb) set logging enabled on
(gdb) run
```

T1 Wrong order of events

"Received before subscribed." "Init ran after first use."

ASK YOURSELF FIRST

- What order should hold, and what guarantees it: a lock, a queue, startup sequence, or luck?
- Which event do I need to see first, second and third?

READING THE RESULT

Write the observed sequence next to the expected one. The first mismatch is the missing ordering guarantee. Fix it with a real synchronization point, not a sleep.

IF YOU OVERSHOOT

rr replay with `reverse-continue` shows what executed before. Without it, rerun and compare logs across runs.

WRITE THIS DOWN

Two lines: expected order, observed order(s).

GDB BOILERPLATE

```
(gdb) dprintf Mgr::init,"T%d INIT\n", $_thread
(gdb) dprintf Bus::subscribe,"T%d SUBSCRIBE\n", $_thread
(gdb) dprintf Bus::onReceive,"T%d RECEIVE\n", $_thread
(gdb) set logging file order.log
(gdb) set logging enabled on
(gdb) run

# Which thread reaches X first?  Gate on the thread:
(gdb) break Bus::onReceive thread 2
(gdb) tbreak Mgr::init thread 1
```

T2 Race condition

"Sometimes stale." "Data torn." "Only fails with two tasks."

ASK YOURSELF FIRST

- What is the shared state, and who writes and reads it?
- Which lock or critical section is supposed to protect it?
- Can I force the bad interleaving by hand?

READING THE RESULT

A read-modify-write without the lock in both threads is the signature. If it only fails when you step in a particular order, you have found the window.

IF YOU OVERSHOOT

rr is the right tool for races, because a recorded failure replays the same way every time.

WRITE THIS DOWN

The sequence of thread, operation, value that reproduces the failure.

GDB BOILERPLATE

```
(gdb) awatch -l shared->count
(gdb) commands
> silent
> printf "tid=%d count=%d\n", $_thread, shared->count
> bt 2
> continue
> end
(gdb) run

# Force an interleaving by hand
(gdb) info threads
(gdb) set scheduler-locking on        # only the thread you step will
run
(gdb) thread 2
(gdb) next                            # advance thread 2 to just
before the read
(gdb) thread 1
(gdb) next                            # let thread 1 write in between
(gdb) set scheduler-locking off

# Find the race automatically, then replay it
$ cmake -DCMAKE_CXX_FLAGS="-fsanitize=thread -g" ...
$ rr record --chaos ./app
```

T3 Hang or deadlock

"It freezes." "Task stopped responding." CPU idle.

ASK YOURSELF FIRST

- Who is waiting on whom?
- For each blocked thread: which lock, queue or semaphore, and who owns it?
- Is it a cycle, or is the owner simply never going to run?

READING THE RESULT

Draw it: A waits for L2 held by B; B waits for L1 held by A. That is a cycle. If the holder is waiting on something else, follow the chain to its end.

IF YOU OVERSHOOT

You cannot reverse a hang, but you can find the first acquisition with rr, or log lock and unlock with `dprintf`.

WRITE THIS DOWN

The wait-for graph, as text: `T1 → L2 (held by T3) → ...`

GDB BOILERPLATE

```
$ gdb -p <pid>                    # attach to the frozen process
(gdb) info threads
(gdb) thread apply all bt             # EVERY stack: the hang command
(gdb) thread 3
(gdb) frame 2                         # the frame that called
lock/wait
(gdb) p *mutex
(gdb) p mutex->__data.__owner         # glibc: kernel thread id of
the holder
(gdb) info threads                    # match that id to the LWP
column

# After a Ctrl-C in a live session the same commands work.
# RTOS: print the semaphore or queue handle and read its holder
field.
```

T4 Starvation and scheduling

"Task never gets CPU." "Low priority task starved." "Why is this running?"

ASK YOURSELF FIRST

- Which task is running now, and what priority?
- Is something spinning, or blocked in a driver or ISR?
- What is the lower task waiting on?

READING THE RESULT

If one function dominates every sample, that is where the CPU goes. Compare with where the starved task is parked.

IF YOU OVERSHOOT

Samples are independent, so there is nothing to reverse. Keep the sample file.

WRITE THIS DOWN

Top 5 sampled frames with counts, and the blocked task's stack.

GDB BOILERPLATE

```
# Poor man's profiler: sample where each thread is, many times
$ for i in $(seq 1 20); do
>   gdb -p <pid> -batch -ex 'thread apply all bt 4' ; sleep 0.2
> done > samples.txt
$ grep -E '^#0|^#1' samples.txt | sort | uniq -c | sort -rn | head

# RTOS: what is running, and at what priority?
(gdb) p pxCurrentTCB->pcTaskName
(gdb) p pxCurrentTCB->uxPriority
(gdb) info threads
```

T5 Deadline or heartbeat timing

"Heartbeat missed." "Timeout fired." "Latency too high."

ASK YOURSELF FIRST

- What is the period and the tolerance, and which clock decides?
- Where is the timestamp taken, and where is it checked?
- Is the producer late, the transport late, or the checker too strict?

READING THE RESULT

Late TX gap: the producer. Late RX with on-time TX: transport or queue. FAULT before the gap exceeds the threshold: the checker's constant or reference time is wrong.

IF YOU OVERSHOOT

Timing does not rewind. Use the log. If you must stop on an event, expect timeouts when you resume.

WRITE THIS DOWN

Three gap series (TX, RX, FAULT) and the threshold they were measured against.

GDB BOILERPLATE

```
(gdb) dprintf Heartbeat::send,"HB-TX tick=%u\n", g_tick
(gdb) dprintf Monitor::onHeartbeat,"HB-RX tick=%u\n", g_tick
(gdb) dprintf Monitor::declareFault,"FAULT tick=%u last=%u\n",
g_tick, last_rx_
(gdb) set logging file hb.log
(gdb) set logging enabled on
(gdb) run

# afterwards, in a shell: gap between consecutive events
$ grep HB-TX hb.log | awk '{split($2,a,"="); t=a[2]; if (NR>1) print
"TX gap", t-prev; prev=t}'
$ grep HB-RX hb.log | awk '{split($2,a,"="); t=a[2]; if (NR>1) print
"RX gap", t-prev; prev=t}'
```

M1 Lifetime: use-after-free, dangling pointer

"Garbage in a field." "Crashes in a destructor." "Works in debug."

ASK YOURSELF FIRST

- Who owns the object and when is it destroyed?
- Who still holds a pointer after that?
- Is it a stack address that escaped?

READING THE RESULT

Fields full of `0xa5a5a5a5` mean you are reading freed memory. The backtrace at the destructor tells you who destroyed it. The holder is who forgot.

IF YOU OVERSHOOT

rr: `watch -l` the object, `reverse-continue` to the free.

WRITE THIS DOWN

Allocation site, free site, and the stale holder. Three stacks.

GDB BOILERPLATE

```
# Make freed memory visible (glibc)
(gdb) set environment MALLOC_PERTURB_=165     # freed bytes become
0xa5
(gdb) set environment MALLOC_CHECK_=3
(gdb) run

# When is it destroyed?
(gdb) break Foo::~Foo if this == 0x5555556a4eb0
(gdb) continue
(gdb) bt
# Who nulls or reuses the pointer?
(gdb) watch -l holder->ptr
(gdb) info symbol 0x7ffe4d2c                  # a stack address? a
heap address?
(gdb) info proc mappings

# Best first tool: rebuild with ASan, it prints alloc and free stacks
$ cmake -DCMAKE_CXX_FLAGS="-fsanitize=address -g" ...
```

M2 Corrupted memory

"foo = 839201382." "Neighbouring variable changed." "Object looks scrambled."

ASK YOURSELF FIRST

- What should the value be, and what do the neighbours look like?
- Who legitimately writes this field? Is the culprit something else (a buffer next door)?
- Does the damage pattern look like ASCII, a counter, or zeros?

READING THE RESULT

A bytewise writer (`memcpy` , `strcpy`) in the watch stack means an overrun from an adjacent object. Compare its length with the destination size.

IF YOU OVERSHOOT

rr or `record` : `reverse-continue` from the corruption to the last write.

WRITE THIS DOWN

Victim address, writer frame, length versus capacity.

GDB BOILERPLATE

```
(gdb) p &var
(gdb) info symbol &var
(gdb) x/16xw (char*)&var - 16          # look at the neighbours
(gdb) x/s &buf                         # ASCII bytes where numbers
should be?
(gdb) ptype/o struct Msg               # layout with padding and
offsets

# The watch catches the writer even if its code never mentions "var"
(gdb) watch -l var if var != 0
(gdb) continue
(gdb) bt                               # memcpy, strcpy, or loop
writer frame
(gdb) frame 1
(gdb) p len
(gdb) p sizeof(buf)
```

M3 Exhaustion: stack, heap, pool, queue

"Works for 20 minutes then dies." "Allocation fails." "Queue full."

ASK YOURSELF FIRST

- Which resource, and what is its capacity?
- Is it a leak (grows steadily) or a burst (spikes)?
- Who consumes and who releases?

READING THE RESULT

Run the same print at three points in time. Growth that never shrinks is a leak; a spike is a burst. Find the producer faster than the consumer.

IF YOU OVERSHOOT

A leak is cumulative, so rewinding does not help. Compare the counters before and after one cycle of the suspect operation.

WRITE THIS DOWN

Resource, capacity, counter values at three times.

GDB BOILERPLATE

```
# Stack depth and use
(gdb) bt -5                            # outermost frames
(gdb) pipe bt | wc -l                  # depth
(gdb) p $sp
(gdb) info frame
(gdb) x/64xw <stack_base>              # painted stack? count
untouched words (0xa5a5a5a5)

# Heap (glibc)
(gdb) call (void)malloc_stats()        # prints to stderr

# Pool, queue, buffer counters
(gdb) p pool.free_count
(gdb) p queue.count
(gdb) p queue.capacity

# Stop where allocation fails
(gdb) catch throw                      # std::bad_alloc
(gdb) break <the failure branch line>
```

B1 Contract mismatch across components

"Each side looks right." "Off by a factor of 1000." "Enum means different things."

ASK YOURSELF FIRST

- What unit, range and meaning does each side assume?
- Who documents it, and does the code agree?
- Is the enum or constant defined twice?

READING THE RESULT

Print the SAME quantity on both sides. A factor of 1000 or a shifted enum name is the entire bug. Check where each side gets its definition.

IF YOU OVERSHOOT

`reverse-step` into the conversion if you passed it. Otherwise breakpoint on the conversion function.

WRITE THIS DOWN

Two lines: "A sends X in unit U", "B expects Y in unit V".

GDB BOILERPLATE

```
# Same logical value, one breakpoint on each side
(gdb) break Sender::send
(gdb) break Receiver::onMsg
(gdb) run
(gdb) p timeout_ms                    # sender: milliseconds?
(gdb) p/d msg.mode ; p msg.mode       # number and name
(gdb) continue
(gdb) p timeout_us                    # receiver: microseconds?
(gdb) ptype enum Mode
(gdb) p sizeof(Msg)
(gdb) p (enum Mode)3
```

B2 Serialization, byte layout, endianness

"Bytes arrive scrambled." "Wrong field offsets." "Works between our boxes, not theirs."

ASK YOURSELF FIRST

- Is the struct packed, padded, or reordered by the compiler?
- What is the byte order on each end?
- Is there a version field, and does the receiver check it?

READING THE RESULT

Match the hex against the spec byte by byte. A hole in `ptype/o` is padding the sender wrote and the receiver did not expect.

IF YOU OVERSHOOT

Serialization is a function, so `reverse-finish` from the first wrong byte back into the writer.

WRITE THIS DOWN

The annotated hex dump: offset, field, value, expected.

GDB BOILERPLATE

```
(gdb) ptype/o struct Msg              # offsets, sizes, holes
(gdb) p sizeof(struct Msg)
(gdb) show endian
(gdb) x/32xb buf                        # raw wire bytes
(gdb) x/8xw buf
(gdb) p/x *(struct Msg*)buf             # reinterpret the bytes
(gdb) dump binary memory msg.bin buf buf+len
$ xxd msg.bin
$ xxd expected.bin | diff - <(xxd msg.bin)
```

V1 Prove my new code runs

"Is my feature actually hit?" "Does this only fire in the scenario I think?"

ASK YOURSELF FIRST

- In which scenario should it fire, and exactly how many times?
- What observable changes: output, state, message?
- Can I force the scenario instead of waiting for it?

READING THE RESULT

Exactly the expected hit count, reached from the expected path, with the expected before and after. A zero or a surprise count is a failure of the change, however the output looks.

IF YOU OVERSHOOT

Overshot? `reverse-finish`, or `tbreak` and rerun.

WRITE THIS DOWN

"Fires N times in scenario S, from path P, changes A to B." This goes in the PR description.

GDB BOILERPLATE

```
(gdb) break MyNewFunction
(gdb) run
(gdb) bt                                # reached from the path I
expected?
(gdb) p state_before
(gdb) finish
(gdb) p state_after
(gdb) info breakpoints                  # "already hit N times":
matches expectation?

# Force the branch if the scenario is rare
(gdb) break Mgr::tick
(gdb) run
(gdb) set var input_flag = 1
(gdb) continue

# Negative test: it must NOT fire here
(gdb) tbreak MyNewFunction if mode == IDLE
```

V2 Before and after (regression)

"It worked before my change." "What behavior changed between A and B?"

ASK YOURSELF FIRST

- What input makes the old and new builds comparable?
- Which values or events summarize behavior?
- Where is the first line where the traces differ?

READING THE RESULT

The first differing line is where behavior forked. Print values, not pointers, since addresses differ between runs.

IF YOU OVERSHOOT

Rerun with `ignore` on the line before the fork and step from there.

WRITE THIS DOWN

before.log, after.log and the diff. Attach all three.

GDB BOILERPLATE

```
# trace.gdb: same script, run against both builds
set pagination off
dprintf Bus::post,"POST type=%d val=%d\n", m.type, m.value
dprintf Tracker::applyDelta,"APPLY %d\n", v
dprintf Mgr::onModeChange,"MODE %d\n", newMode
run
quit

$ gdb -batch -x trace.gdb ./app_old > before.log 2>&1
$ gdb -batch -x trace.gdb ./app_new > after.log  2>&1
$ diff before.log after.log | head -40
```

V3 Impact discovery

"What else uses what I changed?" "Who calls this?"

ASK YOURSELF FIRST

- Which function, field or message am I changing?
- Which scenarios exercise it: the test suite, the sim, a replay?
- Who are the distinct callers or readers?

READING THE RESULT

The sorted list of frame-1 callers is the set of paths your change touches in this scenario. Anything not in the list is a path your testing did not exercise, not proof of safety.

IF YOU OVERSHOOT

Nothing to rewind. Rerun with a different scenario and merge the lists.

WRITE THIS DOWN

Caller list with counts, and which scenarios you ran.

GDB BOILERPLATE

```
(gdb) break ChangedFunction
(gdb) commands
> silent
> bt 3
> continue
> end
(gdb) set logging file callers.log
(gdb) set logging enabled on
(gdb) run                              # run the biggest scenario you
have

$ grep -h '^#1' callers.log | sort | uniq -c | sort -rn
(gdb) info breakpoints                 # total hit count for the
function
```

MISTAKES HAPPEN

Going backwards

You will type `next` one time too many. Plan for it. There are four levels of undo, from cheap to heavy. Pick the cheapest one that still works for your target.

1 · Free, always available

Print before you change, and let GDB remember.

```
(gdb) p total              # $7 = 15  (every print is
saved as $N)
(gdb) set var total = 0    # experiment
(gdb) p $7                 # old value is still there
(gdb) set var total = $7   # put it back
(gdb) p $                  # last value;  $$ = the one
before;  $$3 = three back
(gdb) tbreak <line> ; run # cheapest "rewind" of a short
program: start over
(gdb) info breakpoints     # shows "already hit 37 times"
(gdb) ignore 1 36          # rerun, skip the first 36
hits, stop on #37
```

The hit-count trick is the most useful rewind on targets that cannot record. Crash on hit 37? Rerun with `ignore 1 36` and you land one step before it, every time, if the run is deterministic.

2 · Checkpoint (native Linux, single thread)

```
(gdb) break <Function>
(gdb) run
(gdb) checkpoint           # fork a snapshot of the
process here
(gdb) next                 # try something risky…
(gdb) restart 1            # …and jump back to the
snapshot
(gdb) info checkpoints
(gdb) delete checkpoint 1
```

It forks the process, so it does not capture other threads and does not work on a remote target. Use it to try several `set var` experiments from the same state.

3 · Record and reverse (native, slow)

```
(gdb) start
(gdb) break <a few lines before the suspect area>
(gdb) continue
(gdb) record full          # recording starts HERE, not
earlier
(gdb) next
(gdb) reverse-next         # undo one line
(gdb) reverse-step         # step backwards INTO the call
that just returned
(gdb) reverse-finish       # run back to where this
function was called
(gdb) reverse-continue     # back to the previous
breakpoint or watchpoint
(gdb) reverse-stepi
(gdb) info record
(gdb) record goto begin    # also: end, or an instruction
number
(gdb) record stop          # discard the history, keep
the state
```

Turn recording on late, as close to the problem as you can. It is very slow and uses memory for every instruction. Cap it with `set record full insn-number-max 200000`. Some modern instructions are unsupported and GDB will say so.

4 · Full replay (rr, UDB)

```
$ rr record ./app <args>         # run once, any number
of threads
$ rr record --chaos ./app          # randomize scheduling
to expose races
$ rr replay                        # opens gdb on the
recording
(rr) continue                      # runs to the crash
(rr) watch -l obj->field
(rr) reverse-continue              # stops at the last
write, deterministically
```

rr is Linux (mostly x86) and needs access to hardware performance counters, so it often fails inside VMs. UDB is the commercial option from the talk's sponsor. On a bare-metal target, hardware trace (ETM or a probe vendor's trace tool) is the equivalent.

Which one when?

YOU JUST…	DO THIS
Stepped past the interesting line	If recording: `reverse-next`. If not: `tbreak` the line, `run`, or `ignore` by hit count.
Stepped into a library function you do not care about	`finish` forward, or `reverse-finish` if recording.
Hit a watchpoint, but it is the 3rd write and you need the 1st	`reverse-continue` under `record` or rr. Otherwise add a condition and rerun.
Crashed and want to know who set the bad pointer	rr: `watch -l ptr` then `reverse-continue`. Core file only: you cannot reverse a core, so rerun with a watchpoint.
Changed a variable with `set var` and forgot the old value	`p $` history if you printed first. Otherwise restart.
Want to try three different values from the same state	`checkpoint` before the first one.

DON'T LOSE IT IN THE TERMINAL

Capturing findings

Terminal scrollback is not evidence. Capture at three layers: the raw session, the repeatable script, and the written conclusion.

Layer 1 · Raw session

```
# put in ~/.gdbinit or a project investigate.gdb
set pagination off
set confirm off
set print pretty on
set print object on          # real (derived) type
behind a base pointer
set print elements 200
set logging file gdb-session.log
set logging overwrite off
set logging enabled on       # GDB 12+.  Older GDB:
set logging on
```

```
(gdb) bt full 25                      # on screen
AND in the log
(gdb) pipe bt | wc -l                 # GDB 8.3+:
pipe output to a shell command
(gdb) shell date                      # timestamp a
finding in the log
(gdb) python print(gdb.execute("info threads",
to_string=True))
```

Layer 2 · Repeatable script

```
(gdb) save breakpoints bp.gdb          # every
break, watch, dprintf and condition
$ gdb -x bp.gdb --args ./app <args>    # next
engineer replays your setup
$ gdb -batch -x trace.gdb ./app > run1.log 2>&1   #
unattended run
```

```
# define your own shortcuts once
define where3
  bt 3
  info locals
end
document where3
Short backtrace plus locals.
end
```

A `.gdbinit` in the project directory is ignored unless you allow it: add `set auto-load safe-path /path/to/project` to your user `~/.gdbinit`.

Layer 3 · Snapshots to hand off

```
(gdb) gcore state-at-fault.core         # core of
the live process
(gdb) dump binary memory msg.bin <start> <end>   #
raw bytes of a buffer
(gdb) info registers
(gdb) info sharedlibrary
(gdb) info proc mappings
$ ulimit -c unlimited                    # so the
next crash leaves a core
```

A core file is useless without the exact binary that produced it. Record the git SHA, the compiler flags, and keep the unstripped binary beside it.

Handoff packet checklist

- One-line symptom, expected versus actual
- Exact command to reproduce, and how often it fails
- Build: git SHA, compiler, flags, `-g` present
- `bp.gdb` or `trace.gdb` that reproduces your observation
- Log of the run plus `bt full` at the point of interest
- Hypotheses you eliminated, and the observation that eliminated each
- Root cause and the file:line where the fix belongs

Findings builder

Fill this in as you investigate. It saves in this browser only and turns into Markdown you can paste into the journal, a ticket, or a message to the next engineer.

TICKET AND SYMPTOM

EXPECTED VERSUS ACTUAL

REPRO COMMAND

BUILD

HYPOTHESES (ONE PER LINE, PREFIX WITH + CONFIRMED, - ELIMINATED, ? OPEN)

OBSERVATIONS (COMMAND → WHAT IT SHOWED)

ROOT CAUSE

WHERE THE FIX BELONGS

OPEN QUESTIONS AND NOTES FOR THE NEXT ENGINEER

EASIER ON THE EYES

A better screen than the bare prompt

GDB ships a text user interface. If you want more than that, there are friendlier front ends.

Built-in TUI

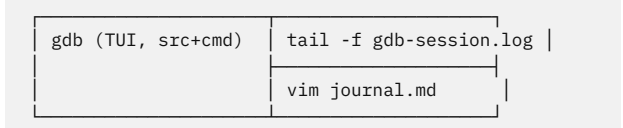
```
$ gdb -tui ./app            # or inside GDB:  tui enable
(gdb) layout src            # source with the current line marked
(gdb) layout split          # source + assembly
(gdb) layout regs           # registers on top
(gdb) focus cmd             # keys go to the command line   (focus src to scroll code)
(gdb) refresh               # redraw if output smears   (or Ctrl-L)
(gdb) tui new-layout dbg src 2 cmd 1   # GDB 10+: your own layout
```

`Ctrl-x a` leaves or enters the TUI, `Ctrl-x o` switches the active window, `Ctrl-x 1` and `Ctrl-x 2` pick one or two windows, `Ctrl-x s` toggles single-key mode (n, s, c, and so on). Pressing Enter repeats the last command, which is how the video steps through a program.

Pick by situation

SITUATION	USE
Over SSH to a lab box	TUI inside tmux. Survives a dropped connection.
Want locals, watches, stack always visible	**gdb-dashboard**: a single `.gdbinit` that draws panes. Send it to a second tmux pane with `dashboard -output /dev/pts/N`.
Local build, want clicks and a watch window	**VS Code** with the C/C++ extension (`cppdbg` uses gdb). For a microcontroller probe, the Cortex-Debug extension.
Browser view of a remote session	**gdbgui**
Emacs user	`M-x gdb` then `gdb-many-windows`

tmux layout for debugging




```
$ tmux new -s dbg \; split-window -h 'tail -f gdb-session.log' \; split-window -v 'vim journal.md' \; select-pane -t 0
```

Make C++ readable

```
(gdb) info pretty-printer          # are the STL printers loaded?
(gdb) p myVector                   # should show {size = 3, ...} not raw _M_impl
(gdb) set print object on
(gdb) set print static-members off
(gdb) set print frame-arguments all
(gdb) set print demangle on
(gdb) p dynamic_cast<Derived*>(basePtr)
(gdb) info vtbl obj                # which virtuals does this object really have
```

YOUR TARGET IS NOT A LAPTOP

Embedded and RTOS specifics

What changes when the program runs on a board, a simulator, or a Linux target. Check which of these applies to your project and skip the rest.

Connect and control

```
# on the target (Linux/RTOS with network):
target$ gdbserver :2345 ./app
host$   gdb ./app
(gdb) target remote 10.0.0.5:2345

# through a probe (OpenOCD shown):
(gdb) target extended-remote :3333
(gdb) monitor reset halt        # "monitor" passes
text to the probe server
(gdb) load                      # flash the ELF
(gdb) continue
(gdb) set remote hardware-watchpoint-limit 4   #
match your core's comparator count
(gdb) show can-use-hw-watchpoints
```

Always load the ELF built with `-g` on the host. The board runs the stripped image; GDB reads symbols from the file.

Optimized builds

If you see `<optimized out>`, rebuild the one file with `-Og -g3`, or the whole target with `-O0` for the investigation.

```
$ gcc -Og -g3 -fno-omit-frame-pointer ...
(gdb) info macro MAX_MSGS          # works with -g3
(gdb) set substitute-path /build/ci/src /home/me/src
# sources moved since the build
(gdb) directory /home/me/src
```

RTOS awareness

With a probe that understands your RTOS (for example OpenOCD `-rtos FreeRTOS`), `info threads` lists tasks instead of one CPU context. Without it, read the kernel's own structures.

```
# FreeRTOS example. Names depend on your port and
config.
(gdb) p pxCurrentTCB->pcTaskName
(gdb) p pxCurrentTCB->uxPriority
(gdb) p uxCurrentNumberOfTasks
(gdb) p xTickCount                  # your own
timeline for deadlines
(gdb) x/64xw pxCurrentTCB->pxStack  # 0xa5a5a5a5 =
stack never touched; count what remains
```

Cortex-M fault (if you are on ARM M-class)

```
(gdb) x/wx 0xE000ED28     # CFSR: which fault
(memory, bus, usage)
(gdb) x/wx 0xE000ED2C     # HFSR
(gdb) x/wx 0xE000ED34     # MMFAR  (valid if CFSR
says so)
(gdb) x/wx 0xE000ED38     # BFAR
(gdb) p/x $lr             # EXC_RETURN: bit 2 set =
frame on PSP, clear = MSP
(gdb) x/8wx $psp          # stacked: r0 r1 r2 r3 r12
lr pc xpsr
(gdb) info symbol *(int*)($psp+24)   # stacked PC =
the faulting instruction
```

Halting changes time. While the target is stopped, hardware timers may keep running and other peers keep talking. A timeout that fires right after you resume might be your breakpoint, not the bug. Use `dprintf` plus the system tick for timing questions, and trust the tick counter rather than wall-clock time.

WORKED EXAMPLE

A bug the compiler cannot explain

Most GDB tutorials crash on a null pointer and the compiler already hinted at it. This one is built to look like real work. It compiles without warnings and the assertion fires in the wrong place: `Tracker` ends up negative, but `Tracker` is not at fault. It is the `practice-sandbox` project already in your handbook. Step through the investigation and watch the hypotheses get crossed off.

H1 · OPEN	H2 · OPEN	H3 · OPEN	H4 · OPEN
Tracker does the arithmetic wrong	Reset was handled in the wrong order	Controller sent an overshooting delta	Something overwrote the memory

1. Symptom

The build is clean. The program runs and then the assertion fires. The failing line is in main, far from whatever went wrong.

```
$ cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug && cmake --build build
$ ./build/sandbox
```

```
total = -10
Assertion failed: (tracker.total() >= 0 && "total must never be negative")
```

Reading it: Nothing here points at a cause. Four plausible ones, so pick the observation that separates them. A watchpoint on the bad value separates H1 and H4 from the other two in one run.

H1 · OPEN	H2 · OPEN	H3 · OPEN	H4 · OPEN
Tracker does the arithmetic wrong	Reset was handled in the wrong order	Controller sent an overshooting delta	Something overwrote the memory

2. Get the live object

A name is ambiguous. An address is not. Run to just after construction and take the address of the field.

```
$ gdb ./build/sandbox
(gdb) break main.cpp:13          # after Bus, Tracker, Controller exist
(gdb) run
(gdb) p &tracker.m_total
```

```
Breakpoint 1, main () at src/main.cpp:13
$1 = (int *) 0x7ffe4d2c3a18
```

Reading it: You now hold the identity of the one variable that matters.

~~H1 · ELIMINATED~~	H2 · OPEN	H3 · OPEN	~~H4 · ELIMINATED~~
~~Tracker does the arithmetic wrong~~	Reset was handled in the wrong order	Controller sent an overshooting delta	~~Something overwrote the memory~~

3. Watch it go bad

Hardware watchpoint, conditioned on the bad value, so the 0 → 10 → 15 → 0 writes do not stop you. The address form avoids a scope problem, since `tracker` is not visible inside `applyDelta`.

```
(gdb) watch *(int*)0x7ffe4d2c3a18 if *(int*)0x7ffe4d2c3a18 < 0
(gdb) continue
```

```
Hardware watchpoint 2: *(int*)0x7ffe4d2c3a18
Old value = 0
New value = -10
Tracker::applyDelta (this=0x7ffe4d2c3a18, v=-10) at src/tracker.cpp:14
```

Reading it: The write came through the legitimate path and added exactly what it was told. H1 is out. A proper writer with a proper argument also rules out H4. The value arrived already wrong.

~~H1 · ELIMINATED~~	H2 · OPEN	H3 · OPEN	~~H4 · ELIMINATED~~
~~Tracker does the arithmetic wrong~~	Reset was handled in the wrong order	Controller sent an overshooting delta	~~Something overwrote the memory~~

4. Read the stack: the sender is gone

The backtrace shows who delivered the message, not who sent it.

```
(gdb) bt
```

```
#0  Tracker::applyDelta (v=-10)        tracker.cpp:14
#1  Tracker::onMessage (m=...)            tracker.cpp:9
#2  <lambda>::operator()                  tracker.cpp:4
#3  Bus::pump                             bus.hpp:26
#4  main                                  main.cpp:19
```

Reading it: Frame 3 is a dispatcher. The message was posted earlier by someone else. Do not read more code. Move the breakpoint to the producer.

~~H1 · ELIMINATED~~	H2 · OPEN	H3 · ROOT CAUSE	~~H4 · ELIMINATED~~
~~Tracker does the arithmetic wrong~~	Reset was handled in the wrong order	Controller sent an overshooting delta	~~Something overwrote the memory~~

5. Break at the producer

Filter on the bad payload so you stop only on the message that matters.

```
(gdb) delete 2
(gdb) break Bus::post if m.value < 0
(gdb) run
(gdb) bt
(gdb) frame 1
(gdb) p *this
```

```
Breakpoint 3, Bus::post (m={type = Delta, value = -10}) at src/bus.hpp:20
#1  Controller::rebalance (target=5)       controller.cpp:12
#2  main                                   main.cpp:18
$2 = {bus_ = ..., believedTotal_ = 15}
```

Reading it: `Controller::rebalance` computed `5 - 15 = -10` from its own `believedTotal_`. H3 is confirmed as the sender. The remaining question is why Controller believes 15.

~~H1 · ELIMINATED~~
~~Tracker does the arithmetic wrong~~

~~H2 · ELIMINATED~~
~~Reset was handled in the wrong order~~

H3 · ROOT CAUSE
Controller sent an overshooting delta

~~H4 · ELIMINATED~~
~~Something overwrote the memory~~

6. Why does it believe 15?

Check whether Tracker was reset before this point.

```
(gdb) break Tracker::reset
(gdb) run
(gdb) p m_total
(gdb) p m_resets
(gdb) bt 2
```

```
Breakpoint 4, Tracker::reset (this=0x7ffe4d2c3a18) at src/tracker.cpp:16
$3 = 15
$4 = 0
#1  Tracker::onMessage (m=...)            tracker.cpp:10
```

Reading it: Reset ran, and it reset Tracker to 0. H2 is out. Controller never learned about the Reset, so its copy of the total went stale. Root cause: two components keep their own copy of one number, and only one hears about the Reset. Fix in Controller (or subscribe it to Reset), not in `applyDelta`.

Command blocks are GDB syntax. Output blocks show the expected shape (adapted from an LLDB run, made-up addresses), not captured GDB output.

Built from `context.txt`, the GDB Beginner Masterclass transcript, and the existing fire handbook chapters (symptom playbook, debugger fundamentals, follow the data, time-travel debugging, memory and concurrency, journal).

Verify every command on your Linux build host before depending on it. Syntax differences between GDB versions are called out where known.